

Compte Rendu de Projet Web

Application Web de Gestion de Notes Personnelles

Technologies utilisees : Laravel API, Laravel Sanctum, React 18, Vite, Axios, MySQL/XAMPP.

Objectif : realiser une application web complete permettant a un utilisateur de creer, consulter, modifier et supprimer ses notes personnelles apres authentication.

Compte de demonstration : demo@example.com / password123.

Element	Description
Backend	API REST Laravel avec authentication par tokens Sanctum.
Frontend	SPA React avec routes publiques et route privatee /notes.
Base de donnees	MySQL via XAMPP, base personal_notes.
Bonus	Recherche de notes et filtre par priorite.

1. Introduction

Le projet consiste a developper une application de type pense-bete personnel. Chaque utilisateur peut s'inscrire, se connecter, puis gerer uniquement ses propres notes. Une note contient un titre obligatoire, un contenu optionnel, une priorite et des dates automatiques de creation et de modification.

L'application respecte le principe d'une SPA : les actions se font sans rechargement complet de page. React communique avec Laravel par requetes AJAX via Axios, et Laravel renvoie des reponses JSON.

2. Architecture technique

L'architecture est separee en deux parties. Le dossier backend contient Laravel, les migrations, les modeles, les controleurs API et les tests. Le dossier frontend contient React, les pages, les composants et la configuration Vite.

Couche	Role
React	Affichage des pages login, register et notes. Gestion de l etat avec hooks.
Axios	Envoi des requetes HTTP vers /api. Ajout du token Bearer aux requetes protegees.
Laravel	Validation, authentification, controleurs REST et logique metier.
Sanctum	Creation et suppression des tokens personnels.
MySQL	Stockage des utilisateurs, tokens et notes.

3. Fonctionnalites realisees

Authentification : inscription avec nom, email et mot de passe confirme ; connexion avec email et mot de passe ; deconnexion avec suppression du token Sanctum.

CRUD des notes : affichage des notes de l utilisateur connecte, creation, modification, suppression avec confirmation, tri par date decroissante cote API.

Interface : badges de priorite, messages de succes ou erreur, formulaire controle, design responsive et navigation protegee.

Bonus : recherche par mot-cle et filtrage par priorite basse, moyenne ou haute.

4. Captures commentees

Les captures peuvent etre realisees depuis le navigateur apres lancement des serveurs. Les ecrans principaux sont les suivants :

Ecran	Commentaire
Inscription	Formulaire public avec nom utilisateur, email, mot de passe et confirmation.
Connexion	Formulaire public qui recupere un token Sanctum apres authentication.
Liste des notes	Page privee affichant les notes de l'utilisateur avec recherche et filtre.
Ajout / edition	Formulaire reutilise pour creer ou modifier une note.
Suppression	Confirmation navigateur avant suppression definitive.

5. Extraits de code

Extrait simplifie du controleur Laravel pour la creation d'une note :

```
public function store(Request $request): JsonResponse
{
    $validated = $this->validateNote($request);
    $note = $request->user()->notes()->create($validated);

    return response()->json([
        'message' => 'Note created successfully.',
        'note' => $note,
    ], 201);
}
```

Extrait simplifie du formulaire React :

```
<input
  type="text"
  name="title"
  value={form.title}
  onChange={onChange}
  maxLength="100"
  required
/>
```

Configuration importante : le frontend utilise Axios et stocke le token dans le contexte AuthContext. Le backend protege les routes notes avec le middleware auth:sanctum.

6. Difficultes et solutions

Difficulté	Solution adoptée
Gestion du token	Stockage du token côté client et ajout automatique dans le header Authorization.
Protection des notes	Vérification de user_id dans NoteController avant show, update ou delete.
Erreurs API	Gestion des statuts 401, 404 et 422 dans React avec messages utilisateur.
Configuration MySQL	Utilisation de .env pour connecter Laravel à la base personal_notes sous XAMPP.
Mise à jour de la liste	Rechargement des notes après création, modification ou suppression.

7. Tests manuels

Action	Résultat attendu	Résultat obtenu
Inscription valide	Compte créé et redirection vers /notes	OK
Connexion demo	Token reçu et accès aux notes	OK
Ajouter une note	Nouvelle note visible dans la liste	OK
Modifier une note	Titre, contenu ou priorité mis à jour	OK
Supprimer une note	Confirmation puis disparition de la note	OK
Titre vide	Blocage de la soumission / erreur validation	OK
Accès sans token	Redirection vers /login	OK

8. Bilan et evolutions

Ce projet a permis de mettre en pratique la creation d'une API REST avec Laravel, la validation des donnees, les relations Eloquent, l'authentification par token avec Sanctum et la communication AJAX avec React.

L'interface React a permis d'utiliser les composants fonctionnels, useState, useEffect, useContext via AuthContext, React Router et Axios. La separation frontend/backend rend le projet clair et evolutif.

Ameliorations possibles : ajout de tags, recherche avantee, partage de notes, export PDF/CSV, pagination et mode sombre.

9. Instructions de lancement

```
Backend:  
cd backend  
copy .env.example .env  
php artisan key:generate  
php artisan migrate:fresh --seed  
php artisan serve
```

```
Frontend:  
cd frontend  
npm install  
npm run dev
```

Le backend fonctionne sur <http://127.0.0.1:8000> et le frontend Vite sur <http://127.0.0.1:5173>.